

Cryptovirology



Use of cryptography to design malicious software (viruses)

- Example: ransomware (aka cryptoworm, cryptovirus)
 - The malware encrypts the user's hard disk using the adversary's public key and asks a fee for decryption
 - It actually happened (e.g., in 2016, to a hospital)

Eee = Redundance

Cryptovirology



Use of cryptography to design malicious software (viruses)

- Example: ransomware (aka cryptoworm, cryptovirus)
 - The malware encrypts the user's hard disk using the adversary's public key and asks a fee for decryption
 - It actually happened (e.g., in 2016, to a hospital)
 - Would it work with symmetric encryption? {...}

↳ No really, even the hospital system has

access to the decryption key ~~to~~ used also to encrypt (design system to keep track of it).

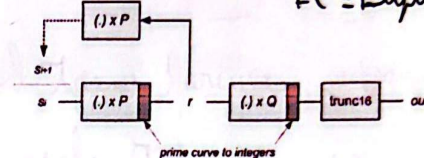
Kleptography

- Branch of cryptovirology
- Study of stealing information securely and subliminally
- Example: modify a public key encryption scheme so that the private key can be derived from the public key
- Academic examples: SETUP attack (Secretly Embedded Trapdoor with Universal Protection)
 - RSA key generation
 - Diffie Hellman key exchange
 - Also applies to protocols (SSL, IPsec)

E. J. Goh, D. Boneh, B. Pinkas, P. Golle, The Design and Implementation of Protocol-Based Hidden Key Recovery, in the proceedings of ISC, pp 165-179, Bristol, UK, October 2003, Springer.

NIST's Dual EC PRNG (I)

EC = Elliptic Curve



- Seed s_0 with P, Q two (public) points of an EC
- EC discrete log: from $s \times P$ it is hard to find s
- What if P, Q were secretly chosen such that: $Q = dP$ and $P = eQ$ (e is d 's inv. mod group order)? {...}

$$nQe = n d P e = n P = \text{Secret State}$$

$$P = (d^{-1})Q = (d^{-1} \times n)Q$$

NIST's Dual EC PRNG (II)

- Snowden leaks suggest NSA backdoored the PRNG
- At least it is possible (and hard to detect)
- Solutions?
 - Truncate more than 16
 - Explain how P and Q are chosen
 - Don't use EC for PRNG (expensive and slow)

Summary

- Public key crypto brings great functionalities but also creates risks & opportunities for adversaries
- Dual EC PRNG highlights the need of transparent and well understood standards (and parameters)
- For example, see the 1M\$ curve proposal
- Or think about the difference between DES and AES

Symmetric Encryption

- One secret key
- Both parties know the key
- $C = \text{Enc}_k(m)$
- $m = \text{Dec}_k(C)$
- Very fast
- Not scalable, one key per user
- AES, DES, 3DES, ChaCha20

Asymmetric Encryption

- Two keys, one public pk and one private sk
- All senders know pk and only one receiver knows sk
- $C = \text{Enc}_{pk}(m)$
- $m = \text{Dec}_{sk}(C)$
- Slower
- One key for all users
- RSA, ElGamal, ECC

In practice:

- Uses Asymmetric encryption to agree on a shared secret key (remained at the end).
- Then use this key for symmetric encryption.

DUAL EC PRNG $\equiv$ A pseudo random number generator based on an Elliptic Curve.
(Standardized by NIST in 2000)

- An elliptic curve E
- Two fixed public points P, Q
- One secret internal state s

Each step:

- 1) Compute $s' = s(n.P)$
- 2) Output part of $s'(n.Q)$
- 3) Update state s'

(Only part of the x -coordinate is output to hide the state.)

Problem if someone knows a secret value $d \mid Q = d.P$ then:

- The generator looks secure even if it is not.
- But whoever chose P and Q , it can be broken.
- With d we can predict all future outputs of the PRNG after seeing a small amount of the output.

Finally this PRNG is not used anymore.

Algorithm Substitution Attacks

- Assume our algorithms cannot be trapdoored
- The next natural step is to backdoor implementations
 - e.g., standard Symmetric Encryption (SE) schemes

Algorithm Substitution Attacks

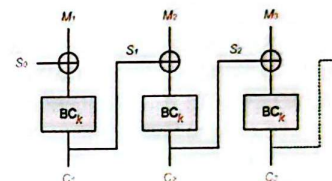
- Assume our algorithms cannot be trapdoored
- The next natural step is to backdoor implementations
 - e.g., standard Symmetric Encryption (SE) schemes
- Algorithmic Substitution Attacks (ASAs)
 - Big brother adversary B substitutes the encryption scheme E by subverted one $\tilde{E}$
 - Typically means the software of E is corrupted
 - Ciphertexts from E and $\tilde{E}$ look similar to reg. users
 - Ciphertexts generated by $\tilde{E}$ leak (e.g., the key) to B

Big Brother $\equiv$ whoever control the implm $\rightarrow$ company
 $\rightarrow$ open-source project owner

Algorithm Substitution Attacks $\equiv$ ASAs

- Assume our algorithms cannot be trapdoored
- The next natural step is to backdoor implementations
 - e.g., standard Symmetric Encryption (SE) schemes
- Algorithmic Substitution Attacks (ASAs)
 - Big brother adversary B substitutes the encryption scheme E by subverted one $\tilde{E}$
 - Typically means the software of E is corrupted
 - Ciphertexts from E and $\tilde{E}$ look similar to reg. users
 - Ciphertexts generated by $\tilde{E}$ leak (e.g., the key) to B
- Can be viewed as a kind of kleptographic attack

Attack against CBC



- $E := (S_0 \xleftarrow{r} \{0,1\}^n, C_i := BC_K(M_i \oplus S_{i-1}), S_i := C_i)$
- How to mount an ASA against this E ?

$S_0 = BC(K) \Rightarrow$ looks the same but enemy can recover K .

Security against ASAs

- Semi-formal definition as follows
 - Detection adversary:** models ordinary users who own K but not $\tilde{K}$ and want to know if ASA is underway
 - Surveillance adversary:** models big brother who owns $\tilde{K}$ but not K and want to read users' traffic
 - Security** if either **detection** adversary trivially succeeds or surveillance adversary miserably **fails**
 - Why is CBC weak against ASAs? $\{ \dots \}$ *Detection of ASAs is hard*
 - Can be formalized as an indistinguishability game

Negative result

- Any randomized stateless SE scheme is vulnerable to an undetectable ASA leading to key recovery
- Proof idea

only attacker knows
 BC and BC^{-1} .

- $BC = \text{Block cipher}$
- $K = \text{master key}$
- 2 messages have different ciphertexts

$$K = BC^{-1}[S_0]$$

ASA $\equiv$ Algorithm Substitution Attacks

we cannot corrupt the algo (maths) so corrupt the software instead.

How it works:

- > Secure encryption scheme E used
- > but powerful adversary replaces with a corrupted version $\tilde{E}$
- > At corruption at implementation ~~for~~ kernel, not in maths

Key properties:

- > For normal users E and $\tilde{E}$ are indistinguishable ($E \sim \tilde{E}$)
- > But $\tilde{E}$ leaks info for attacker such that secrets (or keys)

EX: $\tilde{E}$ biases randomness and encrypts part of the key into ciphertexts, only the attacker can extract it.

$\Rightarrow$ Prop assumption that code is safe and honest.

Cryptology, general concept behind ASA's.

$\hookrightarrow$ Study of crypto algos that seems secure but actually are leaking sensitive data to some parties. $\Rightarrow$ Often achieved by abusing randomness or redundancy. ASA's is a concrete example.

Security against ASA's $\equiv$ Def of what is for a enc scheme to be secure against ASA's.
There are two adversaries, with opposite powers.

1) Detection adversary (the users)

- > Models honest users
- > They know the real secret k
- > They do not the attacker's secret backbone key $\tilde{k}$
- > Goal: Detect if my enc algo has been replaced by a corrupted one?
- > If goal is achieved then the attack fails.

2) Surveillance adversary (big brother)

- > Models big brother
- > They know backbone key $\tilde{k}$
- > They do not know the real encryption key k
- > Goal: Read users' encrypted traffic
- > If big brother (attacker) cannot recover message then the attack fails.

3) Security against ASA's

- > Users detect the substitution of the implm.
- > Big Brother / attacker fails to learn ciphertexts.

4) Cipher Block Chaining (CBC) is weak against ASA's because users cannot see difference between E and $\tilde{E}$ and Big Brother can extract info from $\tilde{E}$.

5) Security as a game: $\rightarrow$ One side tries to distinguish E from $\tilde{E}$
 $\rightarrow$ ASA secure if both cannot distinguish E from $\tilde{E}$ or both cannot decrypt by $\tilde{E}$.

Negative result

- Any randomized stateless SE scheme is vulnerable to an undetectable ASA leading to key recovery
- Proof idea based on the following attack
 - Let $F: \{0,1\}^n \rightarrow \{0,1\}$ be a PRF with key $\tilde{K}$
 - To leak bit $K[j]$ of K , just encrypt with genuine seeds to get $C = E_K(S_0, M)$ until $F_{\tilde{K}}(C, j) = K[j]$ holds

Negative result

If an encrypt^r scheme uses fresh randomness and keeps no state, then it can always be broken without being detected.

- Any randomized stateless SE scheme is vulnerable to an undetectable ASA leading to key recovery
- Proof idea based on the following attack
 - Let $F: \{0,1\}^n \rightarrow \{0,1\}$ be a PRF with key $\tilde{K}$
 - To leak bit $K[j]$ of K , just encrypt with genuine seeds to get $C = E_K(S_0, M)$ until $F_{\tilde{K}}(C, j) = K[j]$ holds
- Main observation: a genuine seed is not enough

Defeating ASAs

- Randomized schemes are now bad...
- But stateless deterministic schemes (in general) cannot achieve semantic security (as usually required)
- Only option: use stateful deterministic schemes such that for any key K , message m and state t of the decryption algorithm there is at most one valid ciphertext.

Instantiation (e.g., for Auth. Enc.)

$E(K, M, A, \sigma)$
 If $(\sigma = 2^t)$ return $\perp$
 $K_1 || K_2 \leftarrow K$;
 $W \leftarrow BC_{K_1}(\sigma || M)$;
 $T \leftarrow F_{K_2}(W || A)$;
 $C \leftarrow (W, T)$;
 $\sigma \leftarrow \sigma + 1$;
 Return (C, σ) ;

$D(K, C, A, \tau)$
 If $(\tau = 2^t)$ return $\perp$
 $K_1 || K_2 \leftarrow K$; $(W, T) \leftarrow C$;
 $(\sigma, M) \leftarrow BC_{K_1}^{-1}(W)$;
 If $(T \neq F_{K_2}(W || A))$ return $\perp$;
 If $(\sigma \neq \tau)$ return $\perp$;
 $\tau \leftarrow \tau + 1$;
 Return (M, τ) ;

$BC \equiv$ Block cipher (deterministic encryption)
 $F \equiv$ Pseudorandom Function (PRF)
 Authenticates ciphertext and meta-data

$K \equiv$ Master symmetric key ($\equiv$ ciphertext)
 $K_1 \equiv$ Encryption key
 $K_2 \equiv$ Authentication key
 $M \equiv$ message
 $A \equiv$ meta-data
 (authenticated but not encrypted)
 Instantiation (e.g., for Auth. Enc.)

Symmetric encryption authenticated scheme
 $0 \equiv$ Sender's counter
 $\tau \equiv$ Receiver's counter
 Instantiation (e.g., for Auth. Enc.)

$E(K, M, A, \sigma)$
 If $(\sigma = 2^t)$ return $\perp$
 $K_1 || K_2 \leftarrow K$;
 $W \leftarrow BC_{K_1}(\sigma || M)$;
 $T \leftarrow F_{K_2}(W || A)$;
 $C \leftarrow (W, T)$;
 $\sigma \leftarrow \sigma + 1$;
 Return (C, σ) ;

$D(K, C, A, \tau)$
 If $(\tau = 2^t)$ return $\perp$
 $K_1 || K_2 \leftarrow K$; $(W, T) \leftarrow C$;
 $(\sigma, M) \leftarrow BC_{K_1}^{-1}(W)$;
 If $(T \neq F_{K_2}(W || A))$ return $\perp$;
 If $(\sigma \neq \tau)$ return $\perp$;
 $\tau \leftarrow \tau + 1$;
 Return (M, τ) ;

- Doubly stateful scheme (σ and τ are counters)

$E(K, M, A, \sigma)$
 If $(\sigma = 2^t)$ return $\perp$
 $K_1 || K_2 \leftarrow K$;
 $W \leftarrow BC_{K_1}(\sigma || M)$;
 $T \leftarrow F_{K_2}(W || A)$;
 $C \leftarrow (W, T)$;
 $\sigma \leftarrow \sigma + 1$;
 Return (C, σ) ;

$D(K, C, A, \tau)$
 If $(\tau = 2^t)$ return $\perp$
 $K_1 || K_2 \leftarrow K$; $(W, T) \leftarrow C$;
 $(\sigma, M) \leftarrow BC_{K_1}^{-1}(W)$;
 If $(T \neq F_{K_2}(W || A))$ return $\perp$;
 If $(\sigma \neq \tau)$ return $\perp$;
 $\tau \leftarrow \tau + 1$;
 Return (M, τ) ;

- Doubly stateful scheme (σ and τ are counters)
- Reject permanently if decryption fails once

Key exfiltration

- Orthogonal concern: assume an adversary who can transmit the key via steganography (despite the previous algorithmic improvements)
- In other words, the key is leaked (but not fully)
- What can be done in this setting?

Adi Shamir (2013): Advanced Persistent Threats have a very narrow pipeline to the outside world...

Steganography = hiding a secret message inside something that looks innocent

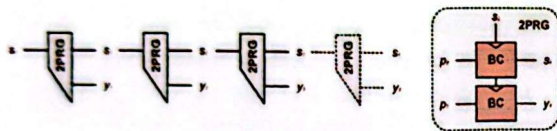
*↳ Hard to detect.
↳ Works even if encryption scheme is perfect.*

Leakage-resilience



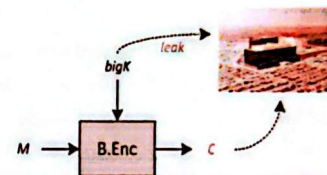
- Each iteration leaks l bits of the running seed
 - See ELEC2760 for the details
- Theorem: if the security of iteration is $\frac{1}{2^{l-1}}$ then security of q iterations is (ideally) $\approx$

Leakage-resilience



- Each iteration leaks l bits of the running seed
 - See ELEC2760 for the details
- Theorem: if the security of iteration is $\frac{1}{2^{l-1}}$ then security of q iterations is (ideally) $\approx \frac{1}{2^{(q \cdot l - 1)}}$
- Problem: initial seed derived from constant master key

Big Key Cryptography (I)



- Typically: k -bit $bigK$ ($k \approx 10^9$), $|leak| \approx k/10$
- One-shot global (i.e., not continuous) leakage bound
 - $\neq$ side-channel attacks against embedded devices

Big Key Cryptography (II)

B.Enc (simplified)

$\{probes\} \leftarrow \{key\ indices\};$
 $K = bigK(probes);$
 $C = BC_K(M);$
 (In fact requires RO)

Big Key Cryptography (II)

B.Enc (simplified)

$\{probes\} \leftarrow \{key\ indices\};$
 $K = bigK(probes);$
 $C = BC_K(M);$
 (In fact requires RO)

- Leads to (surprisingly hard to derive) bound

$$Adv(A) \leq \frac{qt}{2^{p-w(l/k)}}$$

- Where A has data complexity q , time complexity t , and w is the "number of bits of security" per probe

*I = Amount of leaked bits
K = total amount of bits*

S.24

Internal secret state is PARTIALLY leaked. Can we still be secure?

L > Exactly 1 bit, need for k bits

If one iteration has security $\approx 2^{k-1}$:

- > secret has k bits
- > 1 bit is leaked
- > security drops $k-1$ bits

After p iterations $\Rightarrow$ security degrades gradually, but still ok if l is small.

Leakage-resilient crypto tolerates small, repeated leaks - but if the master key never changes, leakage will eventually win.

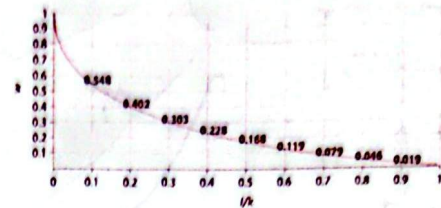
Typically leak is $\frac{1}{10}$ of k .

Big Key Cryptography (III)

- What is $w(l/k)$ worth if $l/k = 0$?
- What is $w(l/k)$ worth if $l/k = 1$?

Big Key Cryptography (III)

- What is $w(l/k)$ worth if $l/k = 0$? 1
- What is $w(l/k)$ worth if $l/k = 1$? 0



probes for 128-bit sec., 1Gbit key, 100Mbit leaked? {...}

Summary

- Key exfiltration with low throughput can be mitigated
- $\exists$ efficient solutions for specific (symmetric) primitives
- But they force designers to re-think differently about how to protect emerging computer systems
- Up to now, focus (mostly) on software and security
- Next (last) threat: hardware Trojans and correctness
 - e.g., what if the manufacturer is an adversary?
 - & triggering events are rare (i.e., detection is hard)
 - e.g., how prevent denial-of-service attacks?

Hardware Trojans are malicious modifications of a hardware design: hidden, activated under specific instructions and leaking information

(Hardware Trojans)

- Ultimate attack where hardware is corrupted at manufacturing time (i.e., malicious manufacturer)
- Can be digital (i.e., trigger and payload via IOs) or physical (e.g., trigger and payload via other signals)
- Example of digital hardware hardware Trojan? {...}

> leak crypte keys
 > disable security checks
 > create covert communication channels
 > Denial of Service

Hardware Trojans

- Ultimate attack where hardware is corrupted at manufacturing time (i.e., malicious manufacturer)
- Can be digital (i.e., trigger and payload via IOs) or physical (e.g., trigger and payload via other signals)
- Example of digital hardware hardware Trojan? {...}
- Example of physical hardware Trojan? {...}

Hardware Trojans

- Ultimate attack where hardware is corrupted at manufacturing time (i.e., malicious manufacturer)
- Can be digital (i.e., trigger and payload via IOs) or physical (e.g., trigger and payload via other signals)
- Example of digital hardware hardware Trojan? {...}
- Example of physical hardware Trojan? {...}
- $\approx$ instantiation of the "device interception issue"

Reactive countermeasures

- Essentially based on detection: device inspection, reverse engineering, physical measurements, ...
- Conceptually unlimited but highly heuristic
 - Become unrealistic for large systems?

Act after an when an attack is detected.
 Detect something suspicious and respond or limit damage.
 → Intrusion detection systems → Key revocation.
 Reactive countermeasures
 → Damage detection → Alarm on failed decryption.

- Essentially based on detection: device inspection, reverse engineering, physical measurements, ...
- Conceptually unlimited but highly heuristic
 - Become unrealistic for large systems?

⇒ Minimizing the parts of the circuit to protect with reactive countermeasures is anyway (highly) desirable

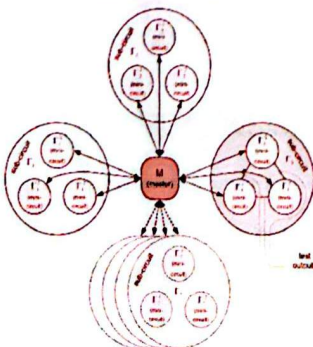
Preventive countermeasures

- Aim to make Trojan insertion/exploitation difficult
- e.g., input scrambling can be used to prevent digitally-triggered Trojans based on "cheat codes"
- How can we compute on scrambled inputs?

Act before the attack happens, adapted design.
 → ASA - resistant encryption schemes
 → Formal verification → secure hardware design
 → Constant-time implementations
 Preventive countermeasures

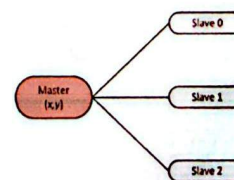
- Aim to make Trojan insertion/exploitation difficult
- e.g., input scrambling can be used to prevent digitally-triggered Trojans based on "cheat codes"
- How can we compute on scrambled inputs? MPC
- Next: how to (formally) prevent cheat codes & time bombs (i.e., digitally-triggered Trojans based on counters) and to guarantee correctness (≠ security)

Trojan-resilience from MPC



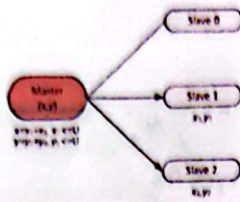
- Target Boolean circuit = XORs (+) and ANDs (·)
- Trusted master
- 3-party computation on sub-circuits (vs. cheat codes)
- Redundancy & functional testing (vs. time bombs)

Initial setting



- Inputs (e.g., x, y) arrive via the (trusted) Master

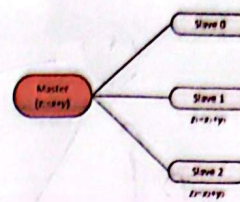
Input sharing



- Assumes Master has access to uniform randomness
- Why does it prevent cheat codes? {...}

Adversary cannot use n_1, n_2

Secure addition (=XOR)

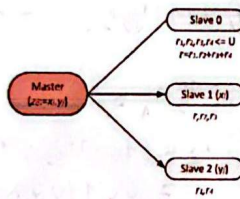


- Can be performed locally by slaves 1 & 2
- Why is it functionally correct? {...}

Both share a portion of the sum: $x_1 + x_2 = \text{all the rest}$

(AND)
||

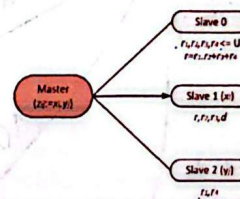
Shares multiplication (I)



- Slave 0 generates "correlated randomness"

Minimize: interaction, randomness and computation.

Shares multiplication (II)

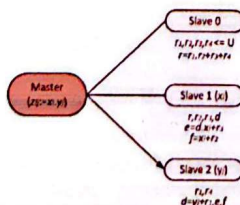


- Slave 2 transmits a masked version of his share
- Why is it secure (in semi-honest setting)? {...}

r is NOT LEAKED.

use XOR outputs on the master!!

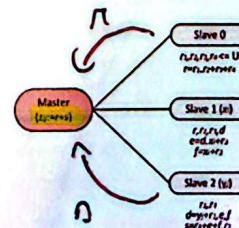
Shares multiplication (III)



- Slave 1 transmits two masked versions of his share

$$\begin{aligned} * \text{ correction: } R &= r_1 + (n_1 + r_1) \cdot n_1 + r_3 + (n_1 + r_1) \cdot n_1 \\ &= r_1 + n_1 \cdot r_1 + n_1 \cdot n_1 + r_3 + n_1 \cdot r_1 + n_1 \cdot n_1 \\ &= n_1 \cdot r_1 + 2r_1 \cdot n_1 + n_1 \cdot n_1 + r_3 + n_1 \cdot n_1 \end{aligned}$$

Shares multiplication (IV) (XOR)

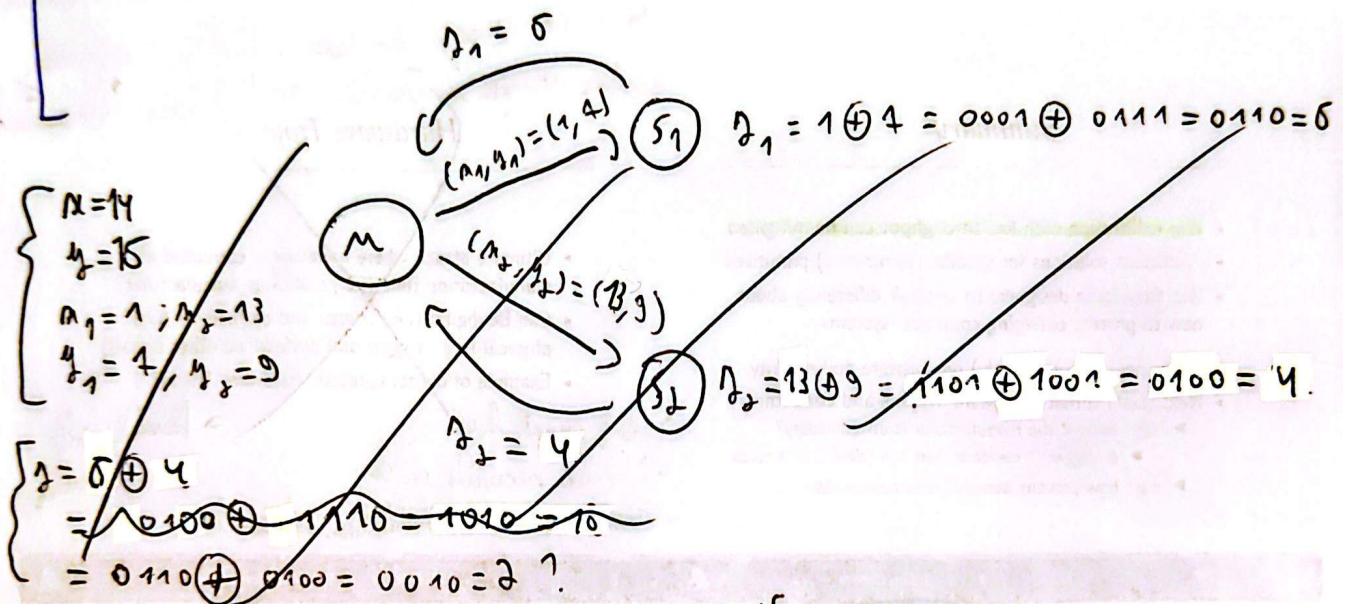
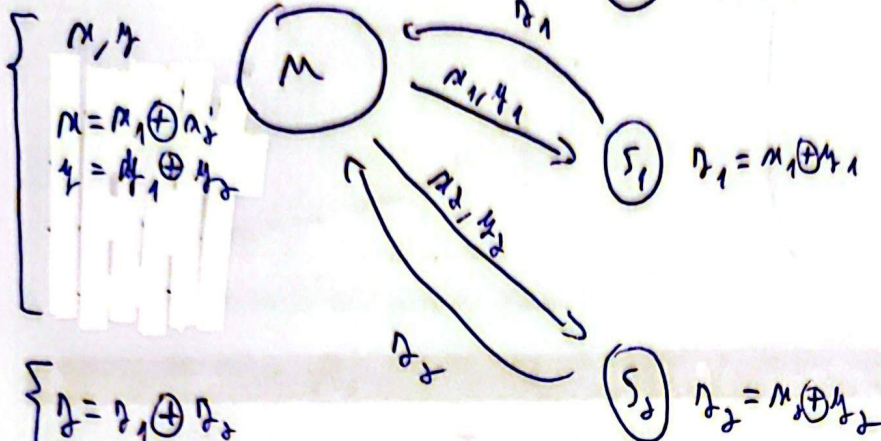


- r and s are the two shares of the partial product
- Why is it functionally correct? {...}

Secure Addition $z = m + y$

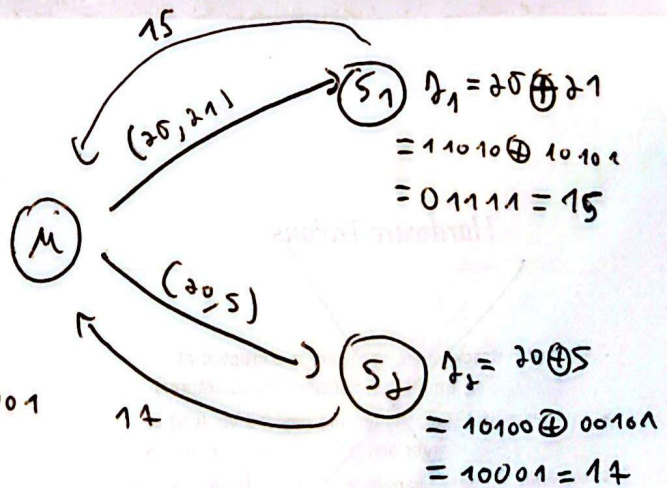
Actually secure XOR

(S_0) (IDLE)



$$\begin{aligned}
 m &= 14 = 01110 \\
 y &= 10000 \\
 y_1 \oplus y_2 &= y \quad (\Rightarrow) \quad 10101 \oplus 00101 = y \\
 m_1 \oplus m_2 &= m \quad (\Rightarrow) \quad 11010 \oplus 10100 = m
 \end{aligned}$$

$$\begin{aligned}
 z &= c_1 \oplus c_2 = 15 \oplus 14 = 01111 \oplus 10001 \\
 &= 11110 = \boxed{30}
 \end{aligned}$$

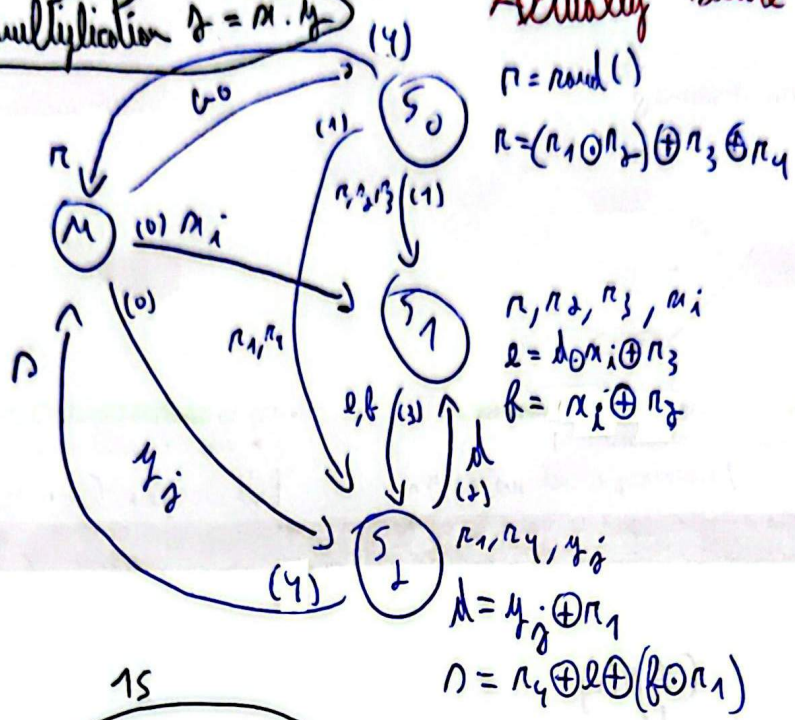


↳ Not addition but XOR just in this case by luck $m + y = m \text{ XOR } y$ by normally $z \neq m + y$ but $\boxed{z = m \text{ XOR } y}$!!

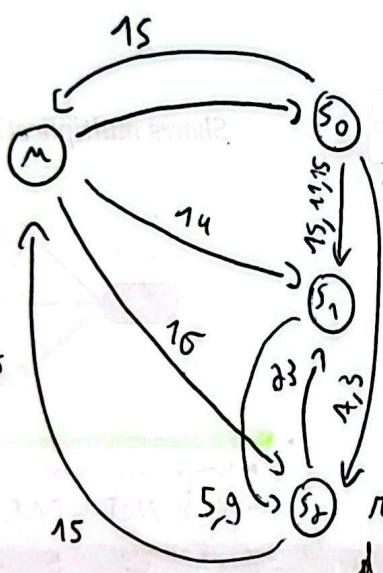
Secure multiplication $\delta = m \cdot y$

Actually secure AND

$\{n_i, y_i\}$
 $\delta = n \oplus \delta$
 y_i



$\{m_i = 14 = 01110\}$
 $\{y_i = 15 = 10000\}$
 $\{n = 15\}$
 $\delta = 15$
 $\delta = n \oplus n = 15 \oplus 15 = 0$



$n_1, n_2, n_3, n_4 = rand(2^4) = 4, 11, 15, 3$
 $n = (4 \oplus 11) \oplus 15 \oplus 3 = (0111 \oplus 1011) \oplus 1111 \oplus 0011$
 $= (0011) \oplus 1111 \oplus 0011 = 1100 \oplus 0011 = 1111 = 15$
 $n = 15, n_2 = 11, n_3 = 15, m_i = 14$
 $l = (23 \oplus 14) \oplus 15 = (1011 \oplus 01110) \oplus 01111$
 $= 00110 \oplus 01111 = 01001 = 9$
 $b = 14 \oplus 11 = 01110 \oplus 01011 = 00101 = 5$
 $n_1 = 4, n_4 = 3, y_i = 15$
 $\delta = 15 \oplus 4 = 10000 \oplus 00111 = 10111 = 23$
 $n = 3 \oplus 9 \oplus (5 \oplus 4) = 0011 \oplus 1001 \oplus (0101 \oplus 0111)$
 $= 0011 \oplus 1001 \oplus (0101) = (1010) \oplus (0101)$
 $= 1111 = 15$

Security vs. correctness

- ▶ Say you only want to produce pseudo-random numbers
- ▶ Then a (much) simpler solution is to
 - ▶ Run λ secure PRGs (without MPC)
 - ▶ XOR their output on the master (which remains secure as long as one PRG output is secure)
- ▶ Probability of a Trojan attack then bounded by $\left(\frac{4n}{k}\right)^k$
- ▶ Useful for the secret sharing rand. (since never output)
- ▶ Keeps the Master simple(er) – i.e., XOR + MAJ gates